

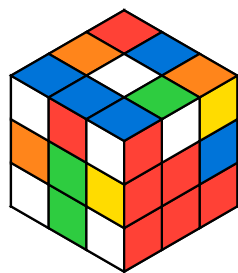
magic-cubes

v0.0.1 MIT

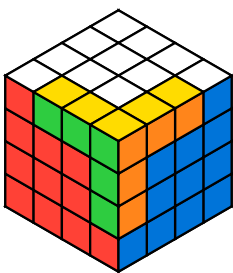
Represent Rubik's cubes of arbitrary size and apply algorithms to them

Rodrigo Alcántara

 @RodAlc24



U' L' U' F' R2 B' R
F U B2 U B' L U' F
U R F'



F U2 L F L' B L U
B' R' L' U R' D' F'
B R2

Table of Contents

About	2
Examples	3
Guide	4
III.1 Creating cubes	4
III.2 Applying algorithms	5
III.2.1 Moves	5
III.2.2 Modifiers	5
III.3 Rendering cubes	6
Full reference	7
IV.1 Custom types	7
IV.1.1 <code>cube</code>	7
IV.1.2 <code>cube-colors</code>	7
IV.2 Functions	7
Examples	10
Index	12

Part I

About

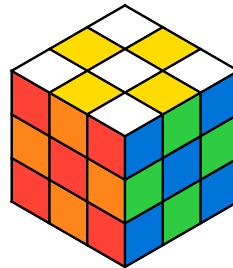
In 1974, Ernő Rubik invented a mechanical puzzle and called it the *Magic Cube*. Years later, in 1980, the puzzle was renamed the *Rubik's Cube*, name by which it is now known. Today, it is considered to be the world's bestselling puzzle game.

[magic-cubes](#) is a package built on top of [CeTZ](#) that allows you to create, manipulate and render Rubik's cubes of any size.

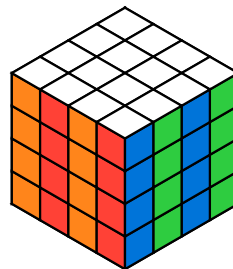
Part II

Examples

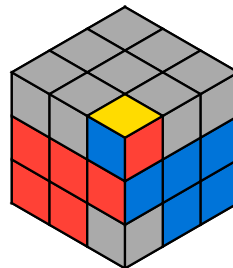
```
#draw_cube(  
  apply(  
    cube(),  
    "F2 B2 R2 L2 U2 D2"  
  )  
)
```



```
#draw_cube(  
  apply(  
    cube(size: 4),  
    "R2 3R2 F2 3F2 R2 3R2"  
  )  
)
```



```
#draw_cube(  
  apply(  
    inverted: true,  
    f2l-cube,  
    "R2 U R2 U R2 U2 R2"  
  )  
)
```



Part III

Guide

III.1 Creating cubes

A cube is represented as a dictionary, and can be easily created with the `#cube` function. It should not be created or modified manually.

The size of the cube can be specified with the `(size)` argument (default is 3). The face colors can be changed with the `(colors)` argument, specifying the faces to change. The default colors are:

```
1 (
2   f: red,
3   r: blue,
4   u: white,
5   b: orange,
6   l: green,
7   d: yellow
8 )
```

It is also possible to specify a face manually, in that case the corresponding value in `(colors)` will be ignored. When specifying manually the colors, all the pieces in the face must be determined.

The order of the pieces in a face is the following:

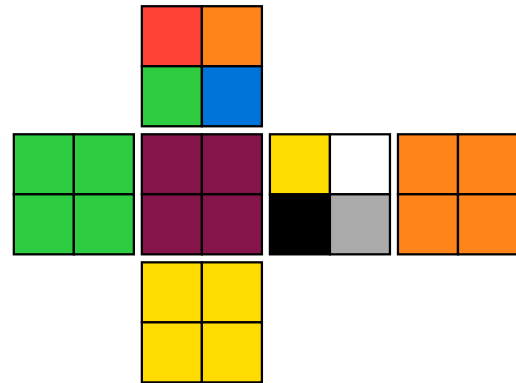
			0	1	2			
			3	4	5			
			6	7	8			
0	1	2	0	1	2	0	1	2
3	4	5	3	4	5	3	4	5
6	7	8	6	7	8	6	7	8
			0	1	2			
			3	4	5			
			6	7	8			

Each face is represented with the default color previously explained, except the up face that is in black.

Note that this way of creating cubes may result in an impossible cube state (you can even use more than six colors!). This can be useful for example to add a gray color to the non relevant pieces.

This order is also used in other size cubes, for example, if we want to create a 2x2x2 cube with a custom color in front face and custom up and right faces we can do:

```
#draw_flat(
  length: 15pt,
  cube(
    size: 2,
    colors: (f: maroon),
    state: (
      u: (
        red,
        orange,
        green,
        blue,
      ),
      r: (
        yellow,
        white,
        black,
        gray,
      ),
    ),
  )
)
```



There also exist some useful cubes predefined:

```
#draw_cube(f2l-cube, length: 10pt)
```



```
#draw_cube(oll-cube, length: 10pt)
```



III.2 Applying algorithms

III.2.1 Moves

The following moves are supported, more will be added in future updates:

- F: front face
- R: right face
- U: up face
- B: back face
- L: left face
- D: down face

All of them represent a single **clockwise** turn in the respective face.

III.2.2 Modifiers

Adding an apostrophe (') or a "2" after the move will cause an anticlockwise turn or a double one (180°) respectively.

III.3 Rendering cubes

Part IV

Full reference

IV.1 Custom types

IV.1.1 `cube`

`cube` is the main type in this package, as it represents the state of a cube. However, you should not create or modify instances manually, as functions such as `#draw_cube` require the cube to be in a consistent state. Instead, use `#cube` to create instances and `parse` to manipulate them.

```
(  
  {size} number  
  {f} array of color  
  {r} array of color  
  {u} array of color  
  {b} array of color  
  {l} array of color  
  {d} array of color  
)
```

The length of the dictionaries must be equal to the square of the size.

IV.1.2 `cube-colors`

This type is used when creating a cube to specify the color of each face.

```
(  
  {f} color  
  {r} color  
  {u} color  
  {b} color  
  {l} color  
  {d} color  
)
```

Not all keys need to be present for a valid `cube-colors` element.

IV.2 Functions

<code>#apply</code>	<code>#draw_cube</code>
<code>#cube</code>	<code>#draw_flat</code>

`#apply({cube}, {alg}, {inverted}: false) → cube-t`

Returns a new cube after applying the algorithm.

Argument	
{cube}	cube-t
The cube to apply the algorithm.	
Argument	
{alg}	str
The algorithm to apply.	
Argument	
{inverted}: false	bool
Whether it should be applied in inverse order.	

#cube(**{size}**: 3, **{colors}**: auto, **{state}**: auto) → cube

This function creates a cube given the size, colors, and/or state. This is the correct way to create cube elements.

The **{state}** argument lets you specify all the pieces (colors) of a face. Note that if a face is specified with this argument, it must be fully specified, and the corresponding color given in **{colors}** is not used.

Argument	
{size}: 3	int
The size of the cube. The default value is 3, i.e., a standard cube.	
Argument	
{colors}: auto	cube-colors auto
Colors of each face of the cube.	
It must be a cube-colors, although it does not need to contain all the keys. In that case, the default color value will be used.	
The default value for each face are: ("f": red)	
Argument	
{state}: auto	
The state of the cube, the default is the solved state.	

```
#draw_cube(
    {cube},
    {x}: 35.264deg,
    {y}: 45deg,
    {z}: 0deg,
    {length}: 20pt
)
```

Draws a cube

Argument —
(cube) cube-t
the cube to draw.

Argument —
(x): 35.264deg angle
the x angle.

Argument —
(y): 45deg angle
the y angle.

Argument —
(z): 0deg angle
the z angle.

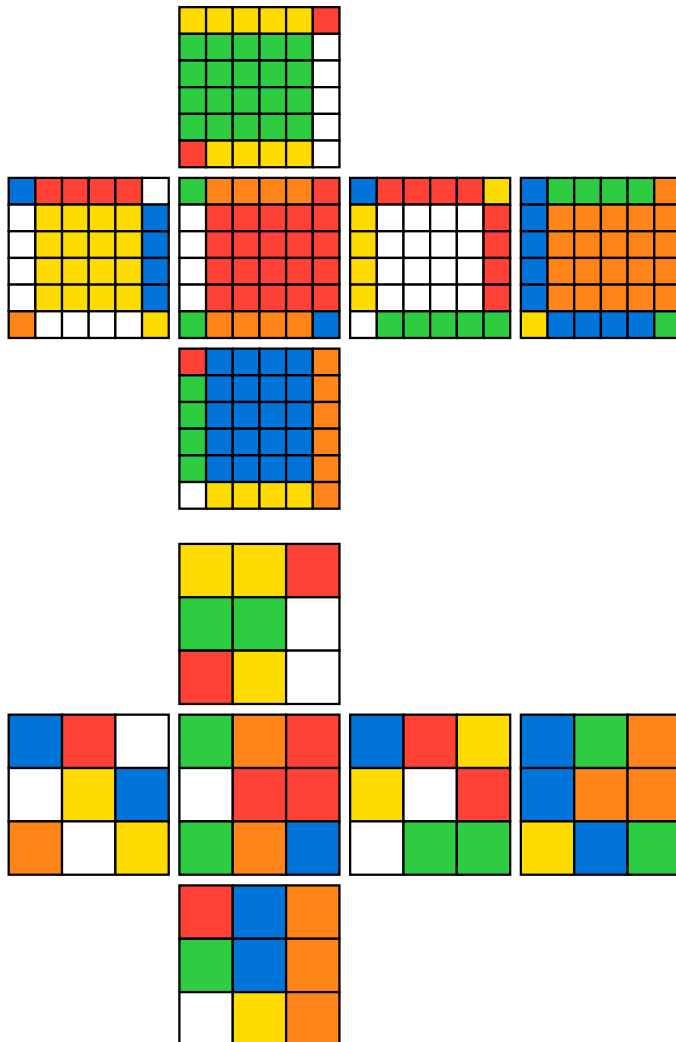
#draw_flat((cube), (length): 20pt)

Draws a flat cube.

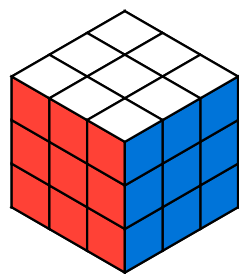
Argument —
(cube) cube-t
The cube to draw.

Part V

Examples



V Examples



,

Part VI

Index

A

`#apply` 7

C

`#cube` 4, 7, 8

`cube` 7

`cube-colors` 7

D

`#draw_cube` 7, 8

`#draw_flat` 7, 9